Information Security Lab
Module 2

Exercise 1

Prof. Shweta Shinde
TAs: Chris Wech

20/21 September 2026

Exercise Plan:

This exercise is **ungraded**

We recommend the following organization for this exercise:

- Before Class:
  - Part 1: 1) - 6)
- During Class:
  - Part 2: 1)
- After Class:
  - Part 2: 2) & 3)
- In your own time (mostly for repetition)
  - Part 1: 7) - 8)

Changelog: There was a mistake in Part 1: 8): The indexing was actually missing the multiplication by pointer size. This is fixed in this version.

Part 1: Physical Memory Organization & Paged Memory

To answer the following questions, assume the **following** fictional CPU characteristics.

- The CPU uses **16-bit** addresses, the Virtual and physical address space is the same size.
- Integer values are interpreted **little endian** (`0x1234` is encoded as the byte sequence `34 12`, from small to large addresses).
- The CPU operates in two modes: Kernel and User
- Paging is supported by a **3-level address translation** (with L1 as root and L3 containing page table entries). The **page offset** given by the **4 least significant bits** of an address.
- Page table entries (only the leafs) may use unused bits to encode the following: `PAGE_PRESENT = 0x1`, `PAGE_USERSPACE = 0x2`, `PAGE_WRITEABLE = 0x8`, and `PAGE_EXECUTABLE = 0x8000`.
- The **`pb`** register holds the physical address of the root of the page table.
- Indexing in each page table array takes 3 bits.

1) How many (byte) addresses fall within every page? How many entries does every intermediate page table array hold?

$2^4 = 16$, $2^3 = 8$

2) How many individual addresses can thus be expressed?

☐ 4096 ■ 8192 ☐ 65536

3) How many of these intermediate page tables (L1 + L2 + L3) exist?

$1 + 2^3 + 2^{3^3} = 1 + 8 + 64 = 73$

4) For a page with the page table leaf entry `0x8631`, mark which flags are set.

■ present ☐ user accessible ☐ writeable ■ executable

5) For a page with the page table leaf entry `0x9f54`, mark which flags are set.

☐ present ☐ user accessible ☐ writeable ■ executable

6) Assume the same flags. For a page with the page table leaf entry `0x153b`, mark which flags are set.

■ present ■ user accessible ■ writeable ☐ executable

Consider the following *physical* memory state to answer the upcoming questions. Addresses not listed in this hexdump can be safely ignored:

```
          0  1    2  3    4  5    6  7    8  9    a  b    c  d    e  f
0x1900: b8 13   85 27   2e 8d   bb 69   ac fd   0f 05   81 6e   d4 6f
0x1910: 15 c2   0c 50   c5 9f   d4 8b   02 e0   f2 91   07 8b   d8 8a
0x1920: 13 f9   81 0b   d7 89   e1 44   7f 3f   6e e3   00 8a   90 28
0x1930: c3 60   0a 1b   68 65   fc a5   5a 3c   31 5f   92 4d   15 cd
0x1940: 7c 5c   ae 0d   1d 0f   1e 36   85 6b   c7 ed   13 07   a8 ec
0x1950: b4 f7   89 22   f3 5f   1d cd   b0 6d   58 79   be 3e   ea bf
0x1960: 6c 91   a3 44   e8 ce   98 c1   9d 7c   e5 20   41 99   68 0b
0x1970: 5e 38   d6 89   25 60   97 5b   a9 c8   df 8e   5b 2f   48 09
0x1980: 6a 19   63 58   f0 1a   76 d3   c0 1a   9b 74   10 96   11 a5
0x1990: 06 a7   a6 19   52 3a   c4 a0   18 e1   3d b2   9e 51   8b 62
...
0x1aa0: e5 73   98 19   de b9   7a 60   02 62   d4 83   f3 a9   7c a2
0x1ab0: b0 56   63 46   23 99   1e 53   df 9a   df 1c   4f 87   ed a6
0x1ac0: 94 bf   40 ff   6a 48   c7 24   92 a7   30 19   d7 50   f8 db
0x1ad0: a9 80   bd 91   1e 84   e9 1c   59 5d   b5 fa   83 97   59 7b
0x1ae0: 0f 44   74 e7   93 03   53 45   d1 72   c0 42   5d f6   0e 26
0x1af0: 86 fc   80 ee   a6 ae   fd b5   60 19   5b 63   b4 55   1f ee
0x1b00: 7f 3b   55 21   90 a8   fd d5   16 f0   68 dd   f6 7d   94 ac
0x1b10: 00 c3   3a 1a   15 66   df 60   6b f4   c8 41   2d 7f   b3 5d
```

The `pb` (address of L1) register is `0x1980`. In the following sub-questions you will be asked to translate the virtual addresses P1 = `0x1293` and P2 = `0xa6c`.

7) Translate the address P1 (`0x1293`).

Virtual frame number: `0x1293 >> PAGE_SHIFT (4) = 0x129`. Each intermediate table holds 8 entries and is indexed by 3 bits, so translate into binary: `0x129 = 0b0001 0010 1001` (remember only up to bit 9 can indicate addresses) and regroup: `100 101 001`. Translate into indices: `{4, 5, 1}`. Each pointer is 2 byte in size so the indices in each intermediate table are `{0x8, 0xa, 0x2}`. L1 table is at `0x1980`. P1's L2 is therefore at `*(1980 + 0x8) = 0x1ac0`. L3 is at index `0xa`, so `*(0x1ac0 + 0xa) = 0x1930`. P1's physical frame | flags is then at `*(0x1930 + 0x2) = 0x1b0a`. To get the physical frame number, get rid of lower 4 bits and upper 3: `0x1b0a >> 4 = 0x1b0`, `0x1b0 = 0b0001 1011 0000`. `0b0001 1011 0000 & 0b0001 1111 1111 = 0b0001 1011 0000 = 0x1b0`. PA is then at `0x1b0 << 4 + 0x3 = 0x1b03` (plus offset).

8) Translate the address P2 (`0xa6c`).

`0xa60 >> 4 = 0xa6 = 0b1010 0110`. Regroup to `0b010 100 110` which gives `{2, 4, 6}`.Multiplying with the pointer size gives `{0x04, 0x08, 0xc}`. `L1 = 0x1980`. `L2 = *(0x1980 + 0x4) = 0x1af0`. `L3 = *(0x1af0 + 0x8) = 0x1960`. PA with flags is `*(1960 + 0xc) = 0x9941`. PFN with flags is `0x9941 >> 0x4 = 0x994`. To clear upper 3 bits: `0x9 = 0b1001 & 0b0001 = 0b0001 = 0x1`. PFN is therefore `0x194` and PA is `0x194 << 0x4 + 0xc = 0x194c`.

Part 2: Integrity in TEEs

We present Cornerstone, a fictional TEE for the `Surety Y` reduced instruction set computing architecture. Cornerstone-I, the first generation, provides protection by encrypting guest memory.

Cornerstone-I encrypts guest memory using *military grade* triple-DES in ECB mode.

1) Attack

You are a malicious cloud service provider hosting Cornerstone-I confidential guests on you hardware. Your goal is to mount an attack that allows you to `ssh` into a confidential virtual machine (cvm).

(a) Does Cornerstone-I's encryption provide guest integrity?

☐ yes ■ no

(b) If the hypervisor makes arbitrary modifications to the guests (encrypted) memory, in which kind of modifications will this result in (for guest virtual memory)?

■ random ☐ controlled

You know that the cvm runs a modified `ssh` daemon, `ssh_e`. For optimization reasons, this program only allows one concurrent connection. As a consequence, its stack is predictable and always at the same guest physical address, which you can predict. The cvm also runs an upload service. You can upload arbitrary data of a full page size and know that it will be page aligned. You can predict which guest physical page will be used for this.

`ssh_e` runs the following code when authenticating a user:

```
bool is_authenticated(struct user u);

bool handle_auth_request(/*[...]*/) {
  // [...]
  bool allowed = is_authenticated(user);
  // if allowed == true the user will be allowed to sign in
  // [...]
}
```

You also know that `is_authenticated` performs the following operation during its epilogue (**`r0` is the result register for `Surety Y`**, while `sp` is its stack pointer):

```
is_authenticated:
  ; [...]
  pop r0
```

and that `sp` points to the last data word (`bool` is stored as an entire word) of a page. You can interrupt the cvm at exactly this moment in time.

(c) Sketch attack that allows an arbitrary user to authenticate

> The encryption is completely deterministic. Therefore the attacker can send a full page (of "true") values and predict where they will end up (by question statement). They can then use the resulting ciphertext of that to replace the page current stack page. Data mangled "below" that should be irrelevant (stack pointer is above it). The result of the method call will be `true`

(d) For which pages can the ciphertext be replayed

- ■ all physical pages
- □ pages mapped to the same `spa`

2) Fixing the issue

Realizing their mistake, Oski (the maintainer of the `Surety Y` specification) introduces a new datastructure to the next generation TEE, Cornerstone-II. This datastructure, called the inverse integrity insurance, or `III` is consulted on ever translation from a (any) virtual address to a host physical address.

This datastructure is a large array in host-physical memory. Each host physical page maps to exactly one entry. The bitfield definition looks like this (you can assume that the sizes make sense):

```
struct III_entry {
  uint64_t guest_physical_frame_number: 36;
  bool host_modifyable: 1;
  uint64_t guest_identifier: 5;
  bool firmware: 1;

  uint64_t host_physical_frame_number: 36;
}
```

The firmware flag means that the page is reserved exclusively for use by the Cornerstone-II firmware. For each of the fields, state if they are required to ensure guest integrity or not, and justify why.
This question is intentionally stated vaguely and should allow for discussion.

(a) `guest_physical_frame_number`

> Yes. If it were note present (but all other fields were) the host could still re-map valid guest pages

(b) `host_modifyable`

Yes, this is the field that stops the host from writing to guest data

(c) `guest_identifier`

Yes or No, This depends if the architecture allows for multiple cvms at the same time. If it does, then this is relevant (otherwise another cvm could *collude* with the malicious host and modify guest memory)

(d) `host_physical_frame_number`

No, This field is redundant because every `struct III_entry` is resolved by its host (system) physical address.

(e) Such a protection scheme requires a secured protection table (in this case the table holding the `struct III_entry` objects.) Propose a scheme to protect the `III` table that **does not** require any memory. If your proposal requires additional invariants, clearly state them)

The `III` table already contains the mappings for *all* physical pages. The CORNERSTONE-II firmware could therefore, for example, assign the pages which contain the `III` to itself by setting the `firmaware` field in the mappings containing them. This requires the firmware to have exclusive access to these pages at some point prior to their use.

3) Another Approach

A competing architecture, `mops` also supports an integrity protected TEE. In contrast to CORNERSTONE-II, however, they decide to ensure guest integrity by computing mac tags for every **cache block**.

4) At which point does the tag *have* to be computed. (write or check)

- ☐ Any Write
- ☐ Any Read
- ■ Writeback/Writethrough from the cache
- ■ Cache Miss on read

5) Name an advantage of this approach

Also works against (naive) ciphertext modifications. Logical integrity protection schemes mostly stops the hypervisor/other cvms from mapping that address, but if they could change DRAM this would not work.

6) Name an disadvantage of this approach

Their performance overhead is larger.